

Stage 1 — Building the Data Layer

Technical Report on Java Architecture, Implementation & Benchmarking Analysis

Course:	Big Data — B.Sc. Data Science and Engineering (ULPGC)	Date:	October 2026
Team:	TheScratchers	Branch:	ft/java-setup (Java 17 target; index benchmark run on JDK 24.0.1)
Author:	Amado Artilles Rodríguez	Repository:	github.com/TheScratchers/stage_1

1. Executive Summary & Scope

This report documents the Java implementation of the Stage 1 data layer for the Project Gutenberg search engine and the results of its three benchmarks: the datalake layouts, the inverted-index structures and the SQLite metadata datamart. The Java module follows the same design as the Python and C++ implementations: a datalake that stores each book as a header file and a body file, a metadata datamart in SQLite, an inverted-index datamart, and a control layer that orchestrates the pipeline one book at a time.

All benchmarks use the shared cross-language contract (`shared/CONTRACT.md`): the same 20 real Gutenberg books, the same tokenizer, the same 10 query words and the same replication rule, at 100, 1,000 and 10,000 books (the metadata benchmark adds a 100,000-book scale). Main findings:

- **Datalake:** the three layouts (time-, book- and batch-based) write at almost the same speed (193–202 books/s at 10,000 books): the cost is dominated by creating two files per book, not by the directory structure. They differ in structure: 2, 10,000 and 10 directories at 10,000 books, with up to 10,000, 1 and 1,000 body files per directory. Incremental detection and recovery work correctly in every layout and scale.
- **Index build:** SQLite is by far the slowest structure to build (111.7 s at 10,000 books, about 45× the monolithic JSON at 2.5 s and 55× the hierarchical JSON at 2.0 s).
- **Index query:** the ranking reverses. Looking up the 10 contract words takes 10.8 ms in SQLite, 1,065 ms in hierarchical JSON and 33.1 s in monolithic JSON. SQLite is about 99× faster than hierarchical JSON and about 3,063× faster than monolithic JSON, because the Java JSON storages re-read and parse their file on every word lookup.
- **Metadata:** inserting is flat at 127–148 books/s up to 100,000 rows (11.3 minutes for 100,000), because every upsert opens, commits and closes its own connection. A primary-key lookup stays at 1.5–1.8 ms at every scale, while the unindexed author search grows to 65 ms (Charles Dickens) and 44 ms (Lewis Carroll) at 100,000 rows.

1.1 Scope and contract compliance

Contract item	Status	How the Java benchmark handles it
Shared dataset (Sec. 1)	Met	Reads the 20 ids from <code>shared/books.txt</code> and tokenizes the real <code>.body.txt</code> files from <code>data/datalake</code> .
Tokenizer (Sec. 2)	Met	Uses <code>TextTokenizer</code> : lowercase, then maximal runs of <code>[A-Za-z]</code> ; no stemming or stopword removal.

Query workload (Sec. 3)	Met	Loads the 10 words from <code>shared/words.txt</code> ; every query operation looks up all 10.
Scales (Sec. 4)	Met	100, 1,000 and 10,000 books in all three benchmarks (JMH @Param for the index); the metadata benchmark adds 100,000.
Replication rule (Sec. 4.1)	Met	Synthetic book at 0-indexed position p uses the real book at position $p \bmod 20$. The benchmark throws an error if any of the 20 books is missing instead of silently shrinking the cycle.
Three index structures	Met	JSON monolithic, hierarchical JSON and SQLite, all behind one <code>InvertedIndexStorage</code> interface.
Datalake metrics (Sec. 6)	Met	<code>DataLakeBenchmark</code> reports write throughput, lookup cost, incremental-processing cost, recovery behaviour and storage overhead (directories, depth, files per directory) for the 3 layouts at 3 scales.
Metadata metrics (Sec. 5)	Met	<code>MetadataBenchmark</code> reports insertion speed, <code>find_by_id</code> and <code>find_by_author</code> (Charles Dickens and Lewis Carroll, timed separately) at 100 / 1,000 / 10,000 / 100,000 books, plus the database size.
Index update (incremental) timing	Not measured	The index benchmark measures bulk build and query only.
Index disk size and file counts	Not measured	Not recorded in <code>results.json</code> .
Peak memory	Partial	JMH's GC profiler reports bytes <i>allocated</i> per operation, which is allocation volume, not peak memory; it is reliable only for the query benchmarks.

2. System Architecture

The code is organised in one package per responsibility, and the index code depends on an interface rather than on a concrete storage, which is what allows the three structures to be benchmarked with identical logic.

Package / class	Responsibility
<code>datalake.Downloader</code>	Downloads a book from Project Gutenberg, cuts it at the START/END OF THE PROJECT GUTENBERG EBOOK markers and stores a <code><id>.header.txt</code> and a <code><id>.body.txt</code> under <code>data/datalake/<yyyyMMdd>/<HH>/</code> .
<code>datalake.DatalakeLayout</code>	The three directory layouts compared in the benchmark (time-based <code>yyyyMMdd/HH</code> , book-based <code><id></code> , batch-based <code>batch_<start>-<end></code> with 1,000 ids per batch) and the helper that writes a book's header and body files.
<code>datamarts.MetadataExtractor</code>	Extracts title, author and language from the header with regular expressions and upserts them into the SQLite table <code>books(book_id, title, author, language)</code> . It also exposes <code>findById</code> and <code>findByAuthor</code> (LIKE '%author%'), each opening its own connection, and accepts a custom database path.
<code>datamarts.TextTokenizer</code>	Contract tokenizer (lowercase, <code>[A-Za-z]+</code>). Covered by 11 JUnit tests.
<code>datamarts.index.InvertedIndexStorage</code>	Interface with <code>save(term, bookId)</code> , <code>build(Map)</code> and <code>search(term)</code> .

<code>JsonMonolithicIndexStorage</code>	One JSON file (<code>inverted_index.json</code>) with every term; Jackson serialization.
<code>JsonHierarchicalIndexStorage</code>	One <code>data.json</code> per first letter of the term, in one folder per letter.
<code>SqliteIndexStorage</code>	Table <code>inverted_index(term TEXT, book_id INTEGER, PRIMARY KEY(term, book_id))</code> through JDBC; one persistent connection; <code>PRAGMA synchronous=OFF</code> and <code>journal_mode=MEMORY</code> ; bulk inserts in JDBC batches of 10,000 rows inside one transaction.
<code>index.InvertedIndexBuilder</code>	Reads a book body, tokenizes it, keeps the unique terms and saves each one through the storage interface.
<code>control.ControlLayer</code>	One pipeline step per run: if a downloaded book is not yet indexed, extract its metadata and index it; otherwise download a random book id (1 to 70,000). Progress is tracked in <code>control/downloaded_books.txt</code> and <code>control/indexed_books.txt</code> . The live pipeline writes the SQLite index.
<code>benchmark.BenchmarkRunner</code>	JMH benchmark class of the inverted index (Section 3.1).
<code>benchmark.DatalakeBenchmark</code>	Datalake layout benchmark (Section 3.2); writes <code>datamarts/benchmark_datalake_results.json</code> .
<code>benchmark.MetadataBenchmark</code>	Metadata benchmark (Section 3.3); writes <code>datamarts/benchmark_metadata_results.json</code> .
<code>benchmark.ContractDataset</code> , <code>JsonOut</code> , <code>DownloadContractBooks</code>	Loader of the 20 contract books (fails fast if any is missing), a dependency-free JSON writer for the result files, and a helper that downloads the missing contract books.

3. Benchmark Methodology

3.1 Inverted index (JMH)

- **Framework:** Java Microbenchmark Harness (JMH) 1.37 on Java HotSpot 64-Bit Server VM 24.0.1, mode *average time* in ms per operation, with JMH's `GCProfiler` attached. Each benchmark runs 1 fork, 1 warm-up iteration of 1 s and 1 measurement iteration of 1 s.
- **Corpus:** the 20 contract books are tokenized once into 20 sets of unique terms. For a given scale, book p (0-indexed) contributes the term set of real book $p \bmod 20$ under the synthetic id $p+1$. The resulting in-memory map (term to list of book ids) is the input of every build.
- **Indexing benchmarks:** measure `build(map)` only (the map is already in memory). The storage folder is cleaned before every iteration, so each iteration starts from an empty storage.
- **Query benchmarks:** the three structures are populated once at trial level, before any measurement. One operation looks up all 10 words of `words.txt`; results are passed to a JMH `Blackhole` so they cannot be optimised away. The per-word figure in the tables is the batch time divided by 10.
- **Memory metric:** `gc.alloc.rate.norm`, the bytes allocated on the heap per operation. It measures allocation churn, not the peak amount of memory in use. It is analysed only for the query benchmarks (Section 4.2): for the build benchmarks JMH's profiler window also covers the per-iteration setup that generates the in-memory map, so their allocation and GC figures in `results.json` mix both costs and are not used here.

3.2 Datalake layouts

- **What is measured:** for each layout and scale, with plain wall-clock timing (`System.nanoTime`): (1) writing N books (header and body file each); (2) 100–200 random lookups, each resolving the path again and reading

the body file; (3) storage overhead (directories, deepest nesting, body files per directory); (4) incremental processing: from a candidate list made of the second half of the existing ids plus $N/10$ brand-new ids, *detect* which are new and then write only those; (5) recovery: wipe the layout, write only the first half (simulated crash), then resume over the full list skipping files that already exist, and check that exactly N body files exist afterwards.

- **Content:** the book at 0-indexed position p gets the header and body of real book $p \bmod 20$ (contract Section 4.1); synthetic ids start at 100,000. Header and body are stripped before writing, like the Downloader does.
- **Choices to keep the run reliable:** a short JVM warm-up pass (200 books per layout, discarded) runs first; the hour directory of the time-based layout is fixed once per run so the benchmark cannot break if the clock crosses an hour boundary; and the synthetic data is written to the system temporary directory, outside the (cloud-synced) project tree.

3.3 Metadata datamart

- **What is measured:** for each scale (100, 1,000, 10,000, 100,000): the time to upsert N rows with `MetadataExtractor.upsertBook` (one connection opened, committed and closed per call, as the production code does, not an idealized bulk insert); the average of 200 `findById` lookups over random ids; 30 repetitions of `findByAuthor("Charles Dickens")` (books 98, 46 and 1400 of the 20, 15% of the rows) and 30 of `findByAuthor("Lewis Carroll")` (book 11, 5% of the rows), timed separately; and the size of the database file.
- **Content:** title, author and language of the 20 contract books are parsed from their real headers (the same regular expressions as the pipeline) and replicated by position ($p \bmod 20$) over synthetic ids starting at 500,000.
- **Built-in check:** after the queries the benchmark compares the number of rows returned for each author with the number expected from the replication rule and aborts if they differ, so a published result implies the queries returned the right rows.

4. Results & Analysis

4.1 Datalake layouts

Write throughput, lookup cost (path resolution plus reading the body) and storage overhead:

Layout	Books	Write (s)	Books/s	Lookup avg (ms)	Directories	Max files/dir
time_based (YYYYMMDD/HH/)	100	0.54	184.7	2.18	2	100
	1,000	5.42	184.3	2.63	2	1,000
	10,000	51.91	192.7	2.84	2	10,000
book_based (<id>/)	100	0.50	201.9	2.04	100	1
	1,000	5.28	189.2	2.01	1,000	1
	10,000	49.47	202.1	2.45	10,000	1
batch_based (batch_N-M/)	100	0.54	185.1	2.18	1	100
	1,000	5.04	198.4	2.23	1	1,000
	10,000	49.52	201.9	2.41	10	1,000

Incremental processing (candidates / new found) and recovery after a simulated crash at 50%:

Layout	Books	Candidates / new	Detect (s)	Write new (s)	Recovery: resumed	Recovery (s)	Complete
time_based (YYYYMMDD/HH/)	100	60 / 10	0.0045	0.070	50	0.24	yes
	1,000	600 / 100	0.0937	0.676	500	2.72	yes
	10,000	6,000 / 1,000	0.5007	5.021	5,000	25.50	yes
book_based (<id>/)	100	60 / 10	0.0050	0.060	50	0.27	yes
	1,000	600 / 100	0.0437	0.591	500	2.59	yes
	10,000	6,000 / 1,000	0.4652	5.120	5,000	25.09	yes
batch_based (batch_N-M/)	100	60 / 10	0.0047	0.062	50	0.25	yes
	1,000	600 / 100	0.0466	0.524	500	2.61	yes
	10,000	6,000 / 1,000	0.4361	5.101	5,000	25.54	yes

- **Writing is I/O-bound and layout-neutral.** Throughput stays between 184 and 202 books/s at every scale and layout, so total time grows linearly with the number of books (about 49–52 s for 10,000). At 10,000 books the time-based layout is the slowest (192.7 books/s against 202.1 and 201.9), a difference of about 5% from a single run per point, too small to rule out run-to-run noise.
- **The layouts differ in structure, not speed.** Time-based keeps 2 directories but puts all 10,000 files in one; book-based creates one directory per book (10,000) with a single file pair in each; batch-based bounds every directory at 1,000 files (10 directories). Lookups cost 2.0–2.8 ms in all cases; the time-based lookup grows slightly with the scale (2.18 to 2.84 ms) while the others stay flatter, which is consistent with a single directory holding many entries, but the effect is small.
- **Incremental ingestion is cheap compared to rebuilding.** At 10,000 books, detecting which of 6,000 candidates are new takes 0.50 s in the time-based layout and 0.47 s in the book-based one (about 0.08 ms per candidate, one existence check each), and writing the 1,000 new books takes about 5 s, against 49–52 s for the full 10,000. Detection therefore costs about a tenth of the write it enables.
- **Recovery is correct and idempotent.** After the simulated crash, resuming wrote exactly the missing half (resumed count = 5,000 at 10,000 books) and left exactly N body files, with no duplicates and nothing lost, in all 9 combinations. Its cost (25.1–25.5 s at 10,000) is about half of a full write, as expected since half of the books are rewritten.
- **Cross-check with Python.** The structural results (directories, deepest nesting, files per directory, number of candidates, new books found and resumed books) are identical to the committed Python results in all 9 layout/scale combinations, which confirms that both implementations apply the same layouts, the same replication rule and the same incremental and recovery procedure. Throughput is of the same order (Python: 201–234 books/s at 10,000 books against 193–202 here), but the two were run on different machines.

4.2 Inverted index: build (bulk, from the in-memory map)

Structure	Books	Build time (s)	vs. previous scale
JSON monolithic	100	0.022	—
	1,000	0.181	×8.0
	10,000	2.50	×13.8
Hierarchical JSON	100	0.067	—

	1,000	0.316	×4.7
	10,000	2.04	×6.4
SQLite	100	0.914	—
	1,000	7.90	×8.6
	10,000	111.72	×14.1

4.3 Inverted index: query (batch of the 10 contract words)

Structure	Books	Batch time (ms)	Per word (ms)	vs. previous scale	Allocated (MB/op)
JSON monolithic	100	598.1	59.81	—	227.3
	1,000	2,879.0	287.9	×4.8	2,194.3
	10,000	33,092.2	3,309.2	×11.5	21,606.6
Hierarchical JSON	100	27.33	2.73	—	12.1
	1,000	107.7	10.77	×3.9	102.1
	10,000	1,064.6	106.5	×9.9	988.5
SQLite	100	1.02	0.10	—	0.02
	1,000	1.93	0.19	×1.9	0.23
	10,000	10.80	1.08	×5.6	2.46

4.4 Metadata datamart (SQLite)

Books	Insert (s)	Books/s	find_by_id (ms)	Dickens (ms)	Carroll (ms)	DB size (KB)
100	0.79	127.3	1.80	2.14	1.75	16
1,000	7.09	141.1	1.57	2.54	2.35	72
10,000	69.58	143.7	1.54	8.38	6.55	620
100,000	675.10	148.1	1.65	65.12	43.84	6,112

- **Insertion is flat per row, so total time is linear.** Throughput stays at 127–148 books/s over three orders of magnitude (0.79 s for 100 rows, 675 s, or 11.3 minutes, for 100,000). Each `upsertBook` call opens a connection, runs one auto-committed `INSERT OR REPLACE` and closes the connection, so a fixed per-call cost (opening the database and committing a transaction) dominates and does not depend on the table size. Loading many rows in a single transaction would be far faster; this benchmark deliberately measures the code path the pipeline actually uses.
- **Primary-key lookups do not depend on table size.** `findById` costs 1.54–1.80 ms from 100 to 100,000 rows. The author query at 100 rows costs about the same (1.75–2.14 ms), which suggests that this floor is mostly the cost of opening a JDBC connection for every call rather than of the query itself (the connection cost was not measured separately).
- **Author search is a full table scan.** There is no index on author and `LIKE '%...%'` cannot use one, so the cost grows with the table once it exceeds the fixed floor: from 8.4 ms at 10,000 rows to 65.1 ms at 100,000 (×7.8) for Charles Dickens and from 6.5 to 43.8 ms (×6.7) for Lewis Carroll. The repeated author is slower at scale (65.1 against 43.8 ms) because it must materialise three times as many rows (15,000 against 5,000), but only 1.49 times slower, because the scan itself is paid in both cases.

- **Storage.** The database grows from 16 KB (100 rows) to 6.3 MB (100,000 rows), about 63 bytes per row at the top scale. The Java table has four columns (no `body_path/header_path`), so its size is not directly comparable with the six-column Python table.
- **Cross-check with Python.** On the same workload Python reports 190 books/s and 526 s at 100,000 rows (Java: 148 books/s, 675 s), 57.4 and 34.6 ms for the two author queries (Java: 65.1 and 43.8), and 0.4–0.6 ms for `find_by_id` (Java: 1.5–1.8). The curves have the same shape (flat insertion, flat primary-key lookup, linear author scan); the Java insertion takes 1.13–1.43× Python's time, the author queries at 100,000 rows take 1.14× (Dickens) and 1.27× (Carroll), and `find_by_id` takes 2.4–3.9×, which is plausibly the JDBC connection cost, but the two runs used different machines, so this is not a controlled comparison.

4.5 Inverted-index analysis

- **Build scaling.** From 1,000 to 10,000 books (10× more data) the build time of the monolithic JSON grows 13.8× and that of SQLite 14.1×, slightly more than linear, which is consistent with larger in-memory structures and bigger files or B-trees. The hierarchical JSON grows only 6.4×, which suggests that at small scales its time is dominated by fixed overhead (creating folders and files) rather than by the amount of data.
- **SQLite pays for being a database on writes.** Even with batched inserts, an in-memory journal and no `fsync`, every (term, book) pair becomes a row that SQLite must place in its primary-key B-tree, whereas the JSON structures serialize the same data as a stream. This is the reason for the 45× gap at 10,000 books.
- **SQLite wins reads by orders of magnitude.** Its lookup goes through the primary-key index and only materialises the matching book ids. Its time still grows with the scale (0.10 ms per word at 100 books, 1.08 ms at 10,000), because with this corpus each term appears in a fixed fraction of the books, so a lookup returns more rows as the scale grows; this is result-set cost, not B-tree depth.
- **Hierarchical JSON is a middle ground.** Because each lookup parses only the file of the term's first letter, it is 31× faster than the monolithic file at 10,000 books, but it still parses a large JSON file for every word (106 ms per word) and allocates about 989 MB per 10-word batch.
- **Practical reading:** JSON is the cheapest structure to *write* in this benchmark and the most expensive to *read*; SQLite is the opposite. For a search engine that builds the index once and queries it many times, the query side dominates, which favours SQLite despite its build cost.

4.6 Limitations and notes for the cross-language comparison

- **Hierarchical is not the same structure in every language.** In Java it is one file per first letter of the term (a few dozen files). The Python and C++ implementations create one file per term (about 35,000 files). Build, lookup and disk figures of the hierarchical structure are therefore not directly comparable across languages.
- **JSON lookups are cold reads in Java.** Each Java JSON lookup reads and parses the file. The Python and C++ benchmarks load the JSON once and then measure in-memory lookups (reported separately as cold-load time and in-memory lookup time).
- **Memory is not peak memory.** Java reports bytes allocated per operation (and only meaningfully for queries); Python reports peak traced allocations and C++ reports process resident memory. The three numbers must not be put in the same column without saying so.
- **Single sample.** Each figure is one JMH measurement iteration (JMH reports no confidence interval). For the fastest cases that iteration contains many operations; for the longest (for example the 111.7 s SQLite build) it contains one. At 100 books the SQLite build lasts under one second, so JMH repeats it inside the iteration and the second repetition probably inserts into rows that already exist (the storage is only cleaned between iterations); that figure is likely slightly optimistic.

- **Datalake and metadata timings are single wall-clock runs.** They are not JMH benchmarks (the runs are too long for JMH) and each figure is one run with no repetitions or confidence interval; a JVM warm-up pass reduces, but does not remove, JIT effects at the smallest scale.
- **Different machines.** Python, C++ and Java were run on different computers with different operating systems and file systems (Java on Windows), so absolute times must not be ranked across languages without that caveat. The result files do not record the CPU, RAM, disk type or JVM version of the machine.
- **Metadata table.** The Java table has no path columns and uses INSERT OR REPLACE instead of the Python ON CONFLICT DO UPDATE; with all ids new this makes no difference to the timings, but the database size differs from Python's.
- **Not recorded for the index:** per-structure disk size, update timing and the vocabulary size. With the contract tokenizer the vocabulary should match the other languages (35,008 terms in Python, 35,007 in C++), but the Java run does not record its own count.

5. Execution & Verification Guide

All commands run from the `java/` directory. The benchmarks need the 20 contract books under `java/data/` `datalake`; if any is missing they stop with an error that lists the missing ids. `DownloadContractBooks` downloads only the missing ones.

```
cd java
mvn clean package                # builds target/benchmarks.jar (shaded, includes JMH and
SQLite)

# 0. Make sure the 20 contract books are present (downloads only the missing ones)
java -cp target/benchmarks.jar
com.thescratchers.searchengine.benchmark.DownloadContractBooks

# 1. Inverted index (JMH, 3 structures x 3 scales) -> results.json
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.BenchmarkRunner

# 2. Datalake layouts (3 layouts x 3 scales) -> datamarts/benchmark_datalake_results.json
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.DatalakeBenchmark

# 3. Metadata datamart (4 scales, up to 100,000 rows) -> datamarts/
benchmark_metadata_results.json
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.MetadataBenchmark
```

The datalake and metadata benchmarks accept an optional comma-separated list of scales (for example `DatalakeBenchmark 100,1000`) and write their synthetic data to the system temporary directory; `-Dbench.dir=<folder>` chooses another location. The metadata run is long (about 11 minutes for the 100,000-row insertion alone) and the SQLite index build at 10,000 books takes about two minutes per JMH iteration. The tables in this report were generated from `java/results.json` (commit `f57697a`) and from the two files in `java/datamarts/` (commit `4beceb0`), produced by the benchmark code of commits `1af44ac` and `c303f8c`.